



APPLIED DEEP LEARNING

04 – ARTIFICIAL NEURAL NETWORKS (BASICS) – II

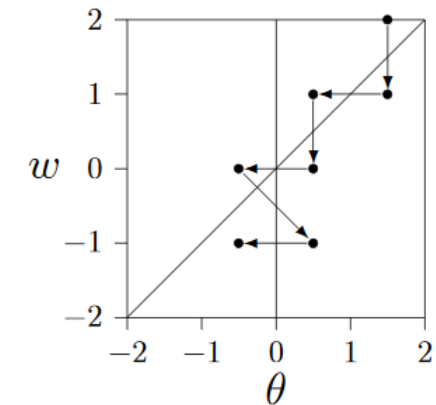
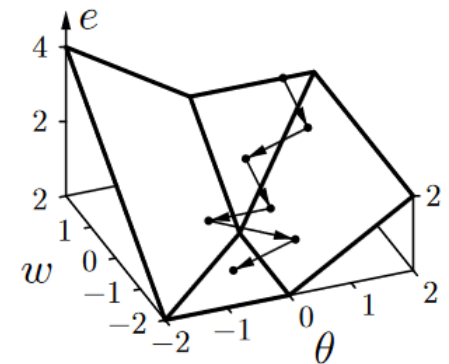
PROF. DR. LARS ACKERMANN-IGL

RECAP: THE DELTA RULE (FOR TLUs)

- Following this direction (of decreasing error value) and the procedure from earlier, weights and threshold can be adjusted to reduce the error
- More formally let $\mathbf{x} = (x_1, \dots, x_n)$ be an input vector for which we know the desired output $\mathbf{o}$. Given $\mathbf{x}$ our (randomly initialized) TLU produces the output $\mathbf{y}$. We can then adjust $\mathbf{w}$ according to:

$$\forall i \in \{0, \dots, n\} \quad w_i^{(new)} = w_i^{(old)} + \Delta w_i, \quad \text{with } \Delta w_i = \eta(o - y)x_i$$

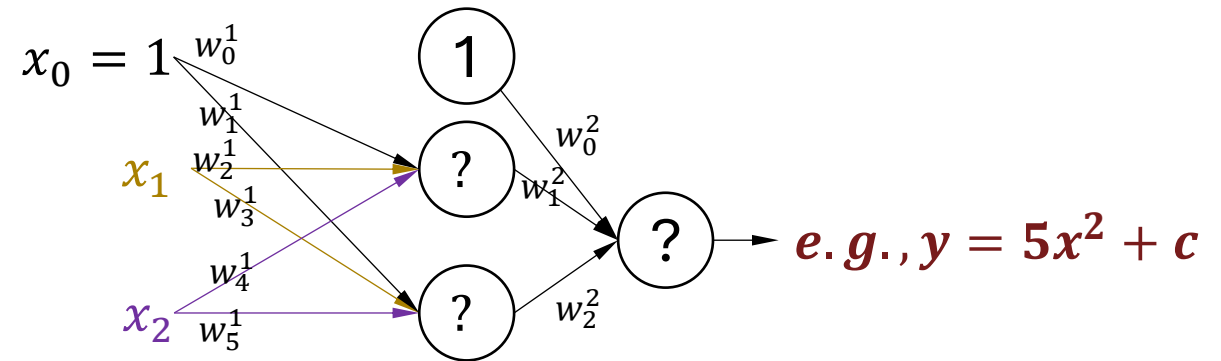
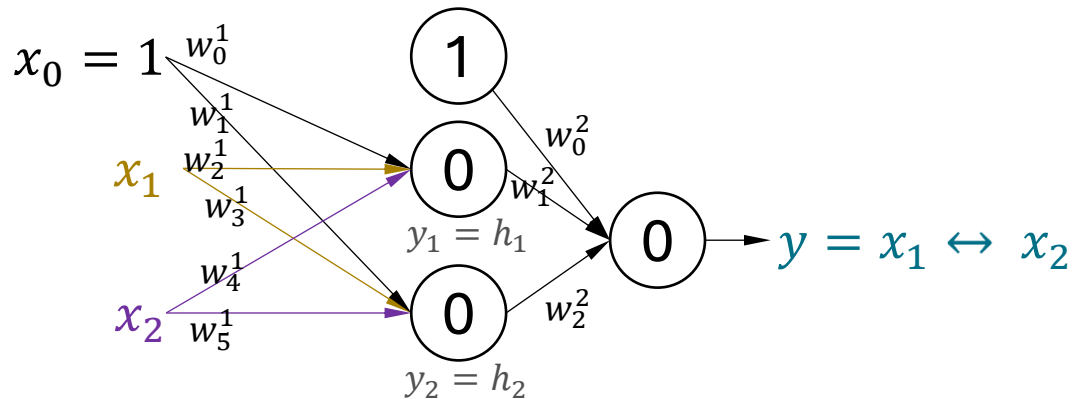
- η (“eta”) is called the **learning rate** and controls the magnitude of change to $\mathbf{w}$
- This procedure is called **delta rule** or **Widrow–Hoff Procedure**



LEARNING PARAMETERS OF AN ANN: LIMITATION OF THE DELTA RULE

$$\forall i \in \{0, \dots, n\} \quad w_i^{(new)} = w_i^{(old)} + \Delta w_i, \quad \text{with } \Delta w_i = \eta(o - y)x_i$$

How to know this value for hidden layers other than the output layer if we are trying to approximate continuous/real-valued functions?

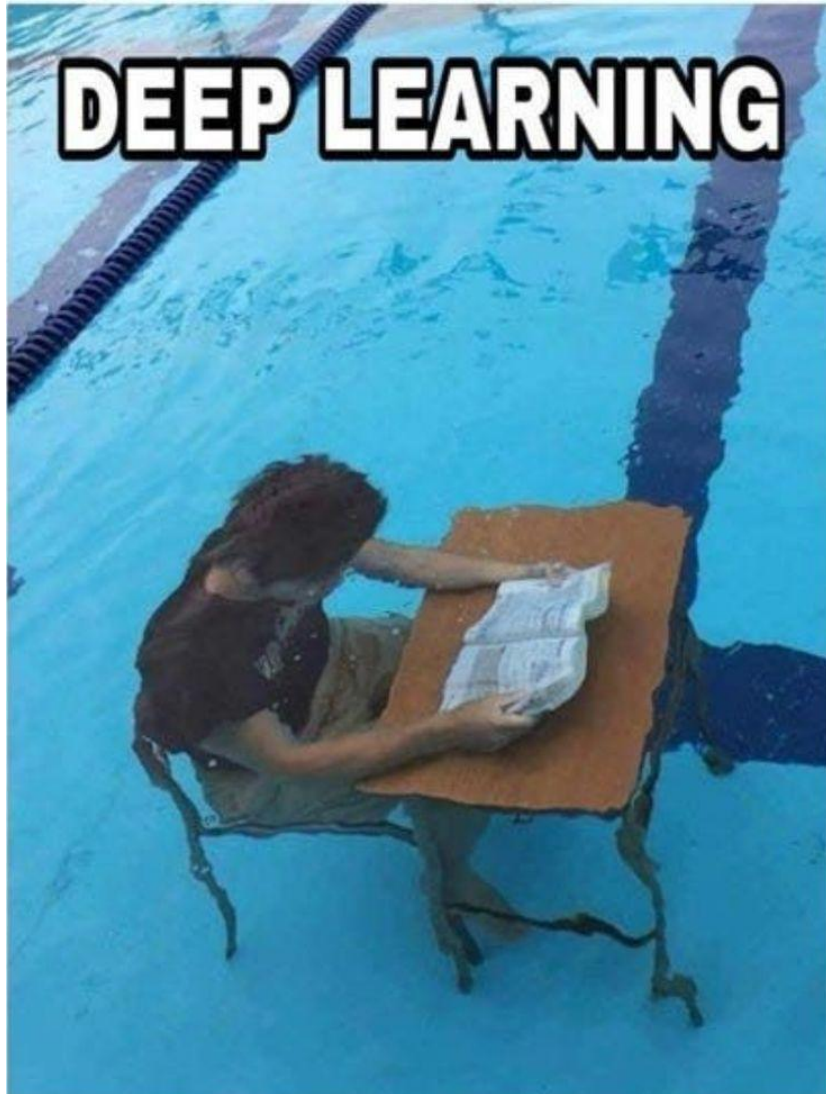


THE DELTA RULE IS FOR SINGLE-LAYER NETWORKS ONLY

- Training data precisely tells, which output should be produced by the output layer for which input.
- But we have no data that tells us what the output of the inner (i.e., the other hidden layers should be).
- We need an algorithm that is able to decide, how it uses the other hidden layers.

This is why we call training artificial neural networks **DEEP LEARNING**!

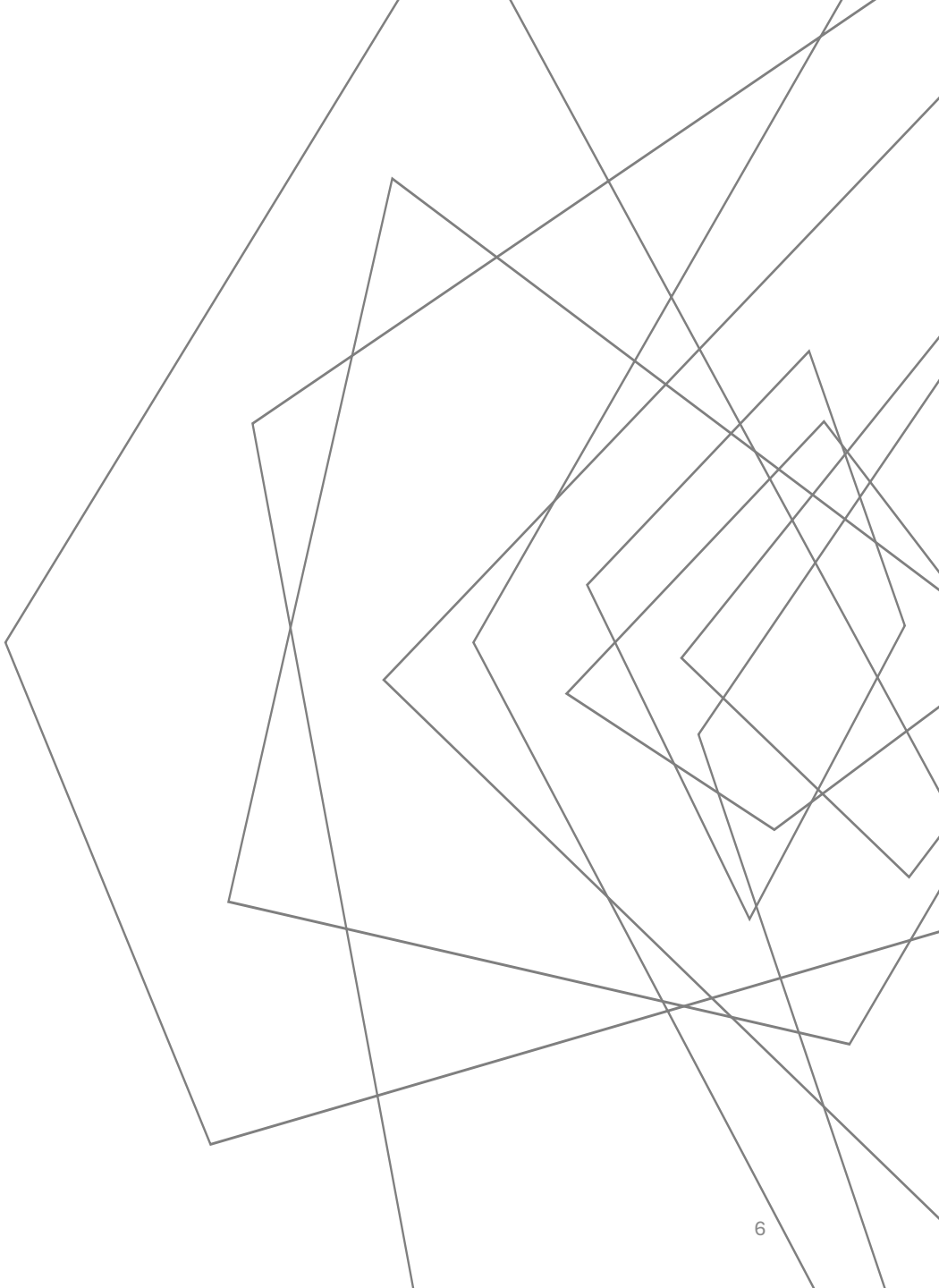
WHAT NOW?



What approaches exist that we can use to adjust weights for hidden layers, i.e., how does deep learning work?

LEARNING APPROACHES

- Input neurons are fixed (size corresponds input size), the output of an Artificial Neural Network depends on
 1. The hidden neurons / layers
 2. Connections between neurons
 3. Weights of connections
 4. The activation function f
- Altering any of those options would alter the resulting output, so **learning** could be achieved by controlled tweaking, called **training**
- Like with threshold logic units, altering **weights** of connections works very well and is the most important method
- Depending on the use-case one of four general training methods can be used
 - **Supervised learning:** for every input, the desired output is known ← Our focus
 - **Unsupervised learning:** outputs are not known, find structure in inputs
 - **Semi-supervised learning:** for some inputs, the desired output is known
 - **Reinforcement learning:** outputs are not known, find reward function for inputs



■ IMPLEMENTING YOUR OWN MLP

- Preprocessing
- Building the model architecture
- Training
- Evaluation

Let's see how we can train an MLP using Keras and supervised learning before we start learning how training works in depth!

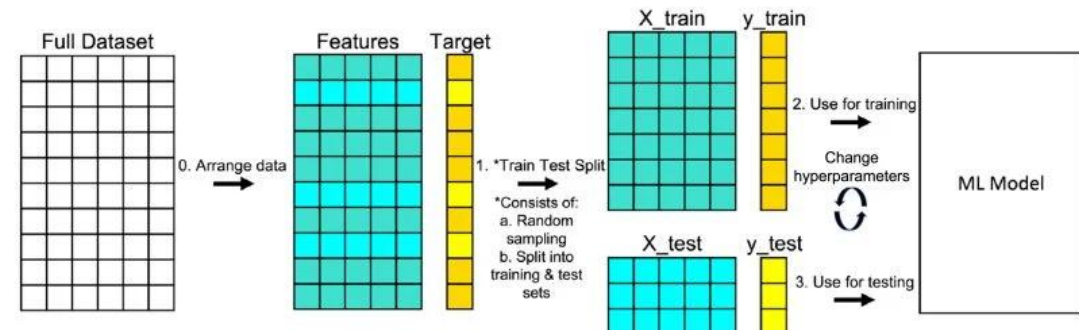
PRE-PROCESSING

- First of all, we prepare the dataset in mainly two steps:
 1. Label encoding
 2. Train-test split
- Label encoding:
 - We cannot pass categorical data to an MLP (but our labels are exactly that)
 - An effective way of encoding such data is one-hot vector encoding:
labels = {'Iris setosa', 'Iris virginica', 'Iris versicolor'}
↓
encoded labels = {[1,0,0], [0,1,0], [0,0,1]}
- Train-test split (recap)
 - Training data: Data used for adjusting model parameters
 - Test data: Unseen (!!) data used to measure the generalization capabilities of the model

```
1 !pip install keras scikit-learn
```

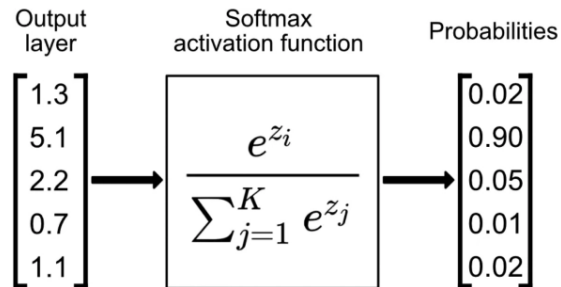
```
1 from keras.models import Sequential
2 from keras.layers import Dense
3 from sklearn.datasets import load_iris
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import LabelBinarizer
```

```
1 # Load dataset
2 iris = load_iris()
3 X = iris.data
4 y = iris.target
5
6 # One-hot encode target labels
7 encoder = LabelBinarizer()
8 y = encoder.fit_transform(y)
9
10 # Split data
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```



BUILDING THE MODEL ARCHITECTURE

- Using the class Sequential we can just add layer after layer, and they are connected exactly like an MLP should be.
- Fully connected layers (aka MLP layers) are called Dense layers in Keras.
- There is a strange activation function used in the last layer: **softmax**. What's that?
- **The short story:** The softmax activation function provides you with a probability distribution over all classes and is a typical last layer in classification models.
- The math:



```
1 # Define MLP model
2 model = Sequential()
3 model.add(Dense(10, input_shape=(4,), activation='relu')) # Hidden layer
4 model.add(Dense(3, activation='softmax')) # Output layer
5 model.summary()
```



- Input dimensionality of 4
- 2 Layers (10 and 3 neurons)
- ReLU and Softmax activation
- More parameters than expected:
 - 50 weights and ...
 - 33 weights ... but why?

TRAINING AND EVALUATION

- When training a model we need to configure:
 - An optimizer: Tells the model how to update its weights during training so it gets better step by step.
 - A loss function: Measures the *error* between the *predicted* and the *expected* results. There are different types of loss functions, such as MSE.
 - Metrics: Tell us how well the model is doing during training, using easy-to-read numbers like accuracy.
- The fit method is implementing all the sophisticated training concepts we will dive into deeper next!
Parameters unknown so far:
 - epochs: How many times the model sees all training data from start to finish.
 - batch_size: How many samples the model looks at at once before updating its weights.

```
1 # Compile model
2 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
3
4 # Train model
5 model.fit(X_train, y_train, epochs=50, batch_size=5, verbose=1)
6
7 # Evaluate model
8 loss, accuracy = model.evaluate(X_test, y_test)
9
10 print(f'Test Accuracy: {accuracy}')
```



Complete notebook will be shared via our Moodle course!



DEEP DIVE

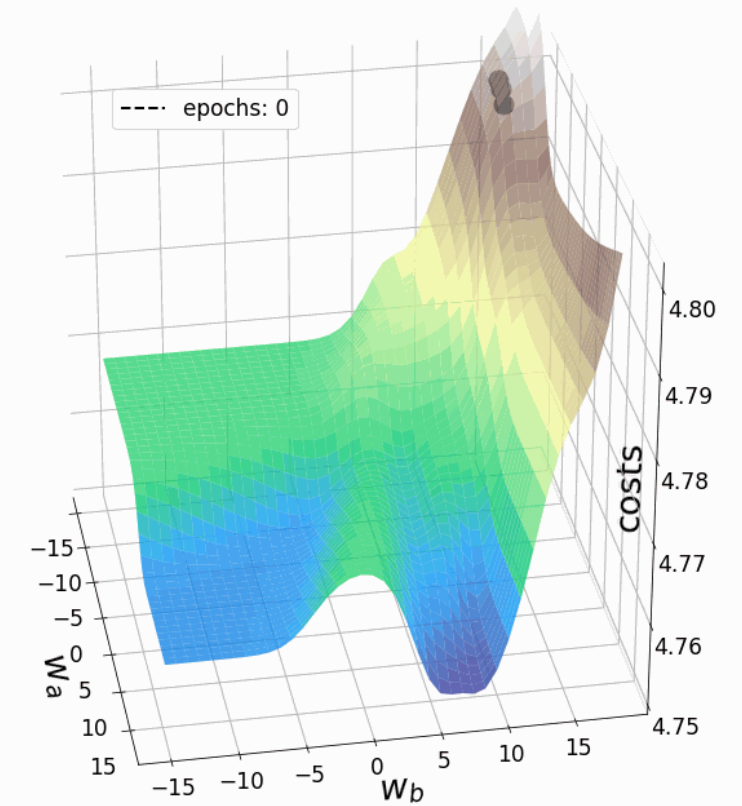
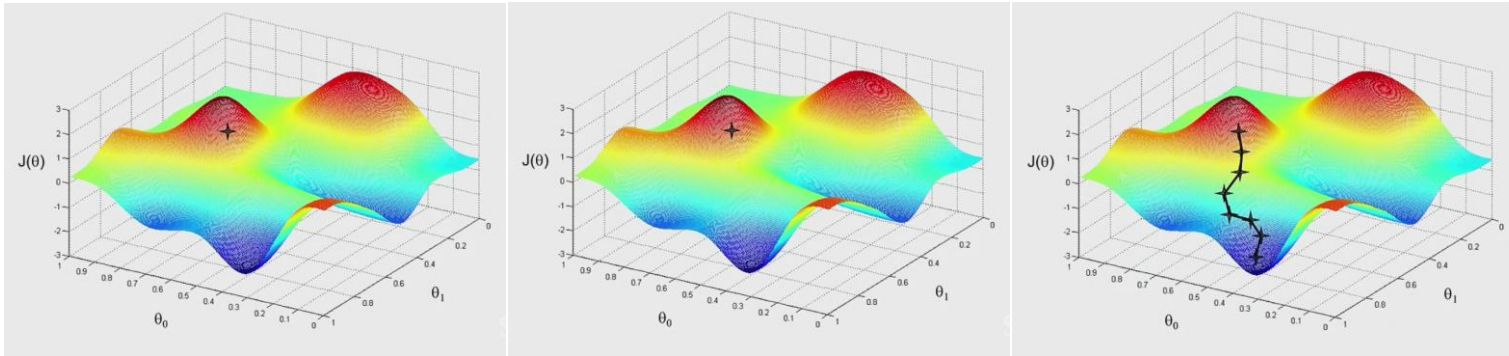
- Gradient Descent
- Backpropagation
- Typical problems with gradient descent

GRADIENT DESCENT

BASIC IDEA

- Starting with a random parameter set w_{init} ...
- iteratively descent into the direction with the maximum (negative) change in the gradient (of the underlying function)
- Repeat until convergence or after a preset amount of steps

Question: What is the gradient of a function?



GRADIENT AND GRADIENT DESCENT (EXPLANATION 1)

GRADIENT OF A FUNCTION

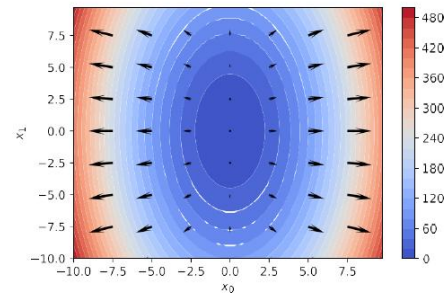
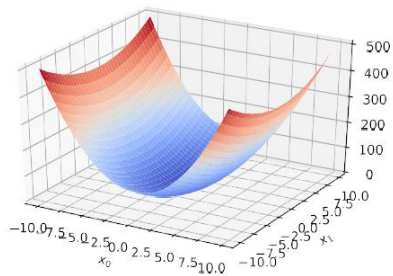
$$p = (x_1, \dots, x_n)$$

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

- Example: Consider the function $F(x_0, x_1) = 4x_0^2 + x_1^2$

$$\frac{\partial y}{\partial x_0} = 8x_0$$

$$\frac{\partial y}{\partial x_1} = 2x_1$$



[6]

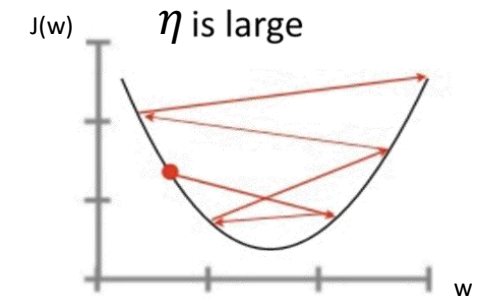
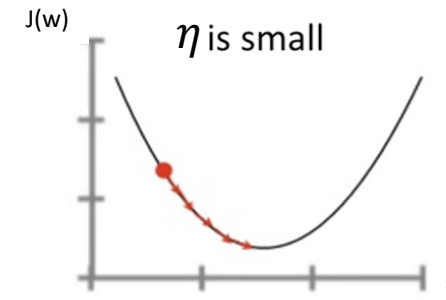
GRADIENT DESCENT PRINCIPLE

- Starting with a random parameter set $w = w_{init}$
- Repeat until convergence or after a preset amount of steps:
 - iteratively descent into the direction with the maximum (negative) change in the gradient

$$w := w - \eta \nabla J(w)$$

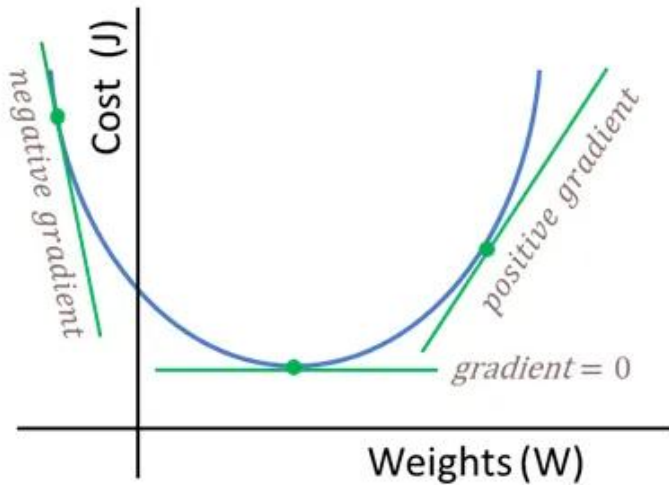


- This means that we derive the error function after each individual weight.
- Impact of learning rate η



Parameter η has an influence on the convergence of the gradient descent.
In practice, $\eta = 0.01$ is often a good start parameter (with normalized / scaled features)

MEANING OF THE GRADIENT



Figure*: Costs depending on choice of weights

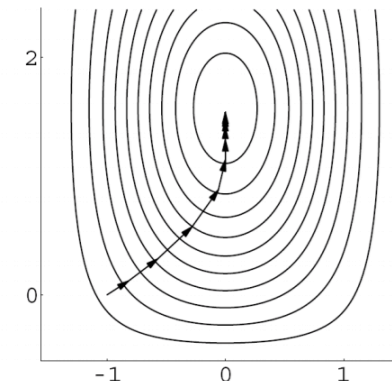
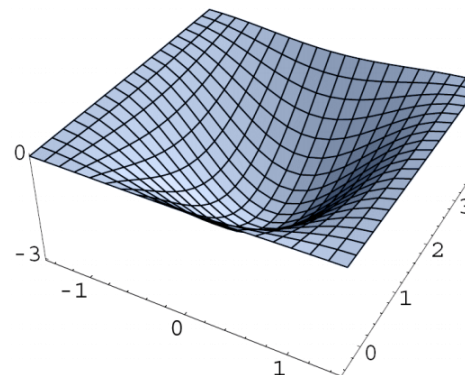
If the gradient is ...

- positive = increasing w means increasing loss,
- negative = increasing w means decreasing loss,
- zero = changing w (slightly) does not affect the loss (at that point, i.e. in a region close to the current weight values)

GRADIENT AND GRADIENT DESCENT (EXPLANATION 2)

- As the output of our neural network for a given input depends on the n connection weights, so does the error
- Minimizing this error is therefore reduced to the problem of minimizing the n -dimensional error function, which is commonly done with **gradient descent** methods
- The **gradient** is a vector g defined at every differentiable point of a function. It points in the **direction of the steepest ascent**, and its norm $\|g\|$ indicates how steeply the function increases in that direction.
- Accordingly, the **negative gradient** ($-g$) points exactly towards the **steepest descent**
- **Gradient descent** means adjusting a point $\mathbf{p}$ iteratively according to the gradient $\nabla f(\mathbf{p})$ of a **differentiable** function f
- This can be visualized as a ball, rolling downhill, until it finds a valley (minimum)

Does this work
with our TLUs?



COMING BACK TO TLUs – AND THEIR STEP FUNCTION

GRADIENT DESCENT REQUIRES DIFFERENTIABLE FUNCTIONS

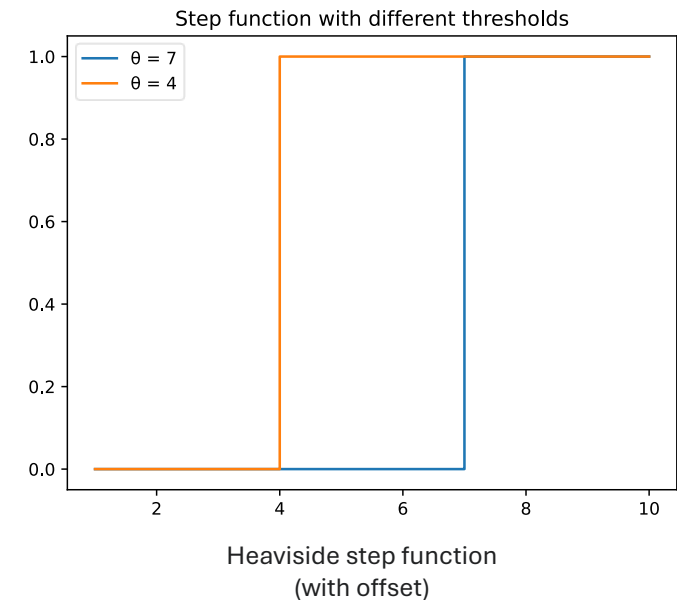
➡ $\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$

THE STEP FUNCTION HAS NO DERIVATIVE

(in its original sense; there is a so-called distributional derivative: The Dirac Delta function, which does not help as well)

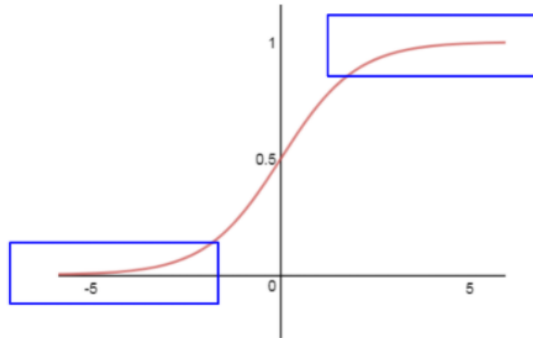
- The derivative of the step function is 0 for all $x \neq 0$
- For $x = 0$ the derivative is undefined

So what can we do now?



RECAP: DIFFERENTIABLE ACTIVATION FUNCTIONS

Sigmoid function: $f(x) = \frac{1}{1 + e^{-x}}$

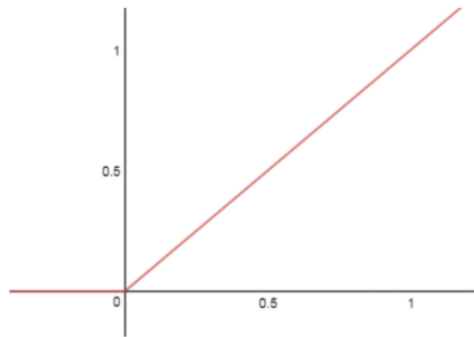


- **Gradient problem**

- Not centered around 0

- **complex** in computation

Rectified Linear Unit (ReLU): $R(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$

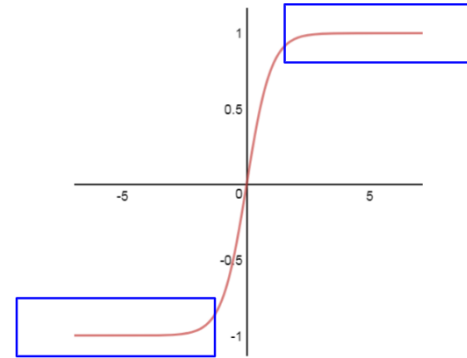


+ very easy to compute (in comparison to sig or tanh)

+ Gradient problem solved

o it can happen, that there are neurons in the NN which will never be activated (“dead neurons”)

Tangens hyperbolicus (tanh) function $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

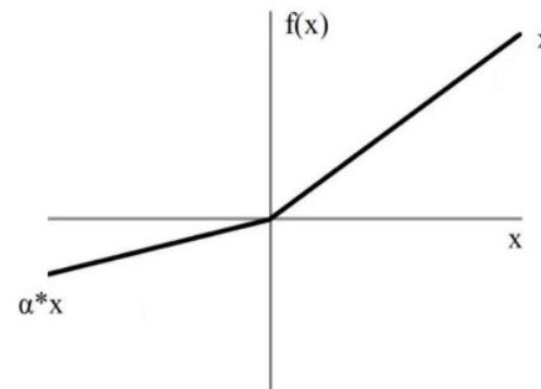


+ same properties as sigmoid, but in addition **centered around the value 0**

- **Gradient problem still exists**

- even **more complex** than sig

Leaky ReLU: $R(x) = \begin{cases} x & \text{if } x \geq 0 \\ \alpha x & \text{otherwise} \end{cases}$

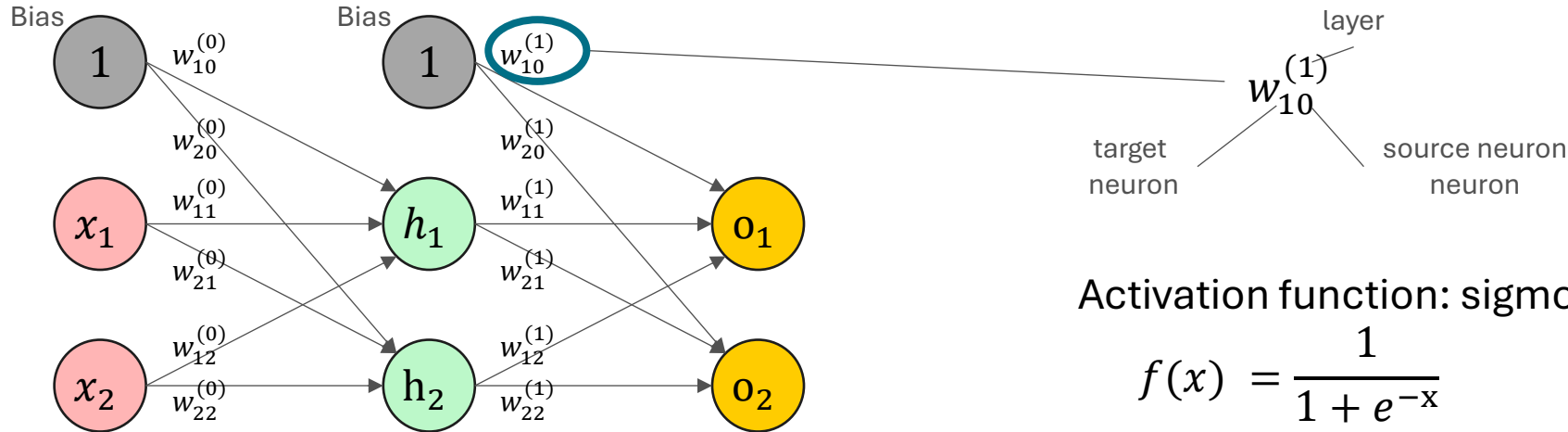


+ solves the “problem” of “dead neurons”

BACKPROPAGATION – OVERVIEW

Gradient descent is the method that **updates the weights**, and backpropagation is the procedure that **computes the errors and gradients** needed for those updates **efficiently**.

- Consider this simple MLP network consisting of an input layer, a hidden layer and an output layer

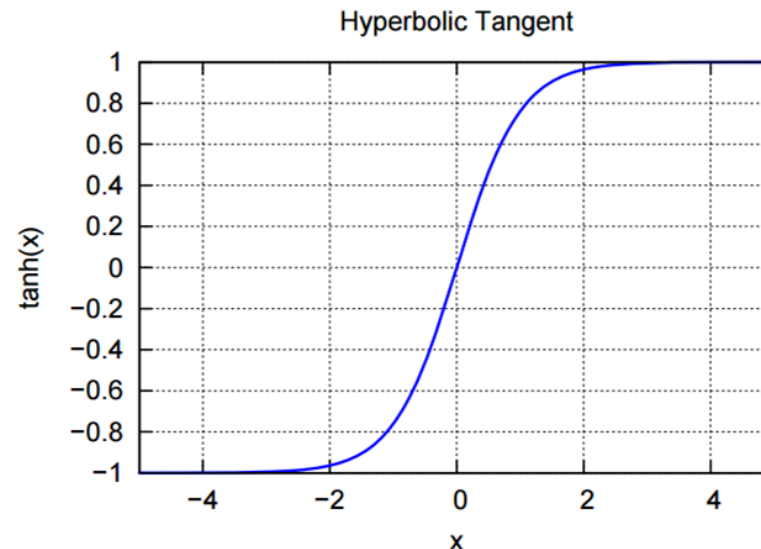


- Our training data has samples (called **patterns**) of two input signals (x_1, x_2) and a corresponding desired output (target) value y
- To train this network we can use the **backpropagation algorithm**
 1. Initialize weights of our network
 2. Calculate the network output for a given pattern (**forward pass** (sometimes also: **forward propagation**) -> SEE PREVIOUS LECTURE)
 3. Calculate the error for that output and the corresponding gradient
 4. Use error to adjust weights (**gradient descent via error backpropagation**)
 5. **Repeat** from step 2 until error is less than a given threshold or until a given maximum number of iterations is reached

BACKPROPAGATION – 1. INIT WEIGHTS

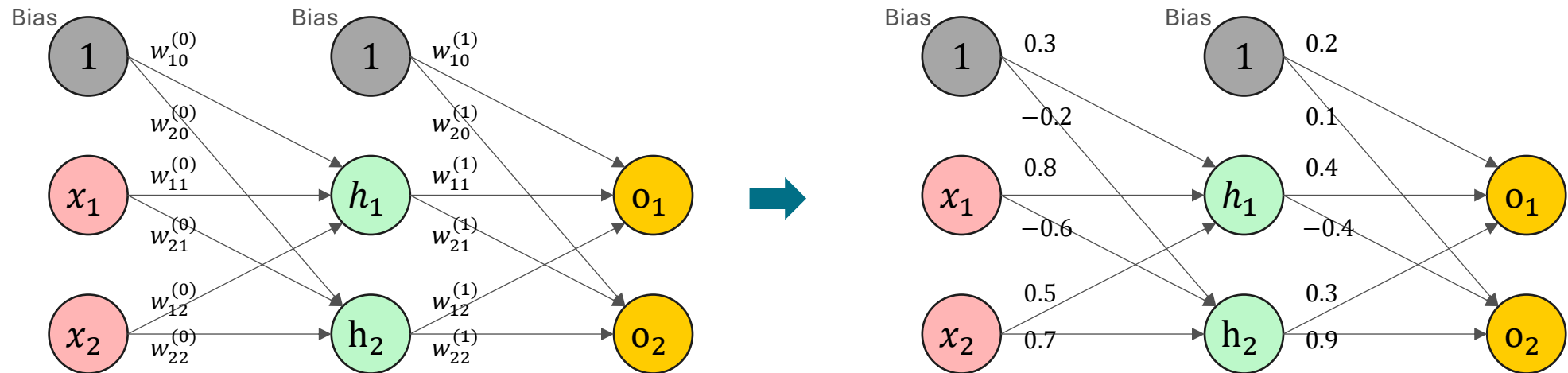
Randomly initialize all connection weights

- This step can be viewed as finding a **random starting point** on the error surface (but from which distribution should we draw?)
- Choosing the **correct distribution** for initial weights is important for convergence, training speed and final accuracy
- It's heavily dependent on the activation function used (e.g. *tanh* can't cope with large weights, producing near **identical values** for large inputs, making training very hard)
 - Recommendation for this activation function (assuming normalized/standardized inputs): [-4; 4]



FOR THE REMAINDER OF THE EXPLANATION ...

- ... we will work with an example and concrete numbers (important: repeat the calculation at home and try to generalize it!)



Step 1 done: Initialized with random weights

2. CALCULATE THE FORWARD PASS (NOTHING NEW)

here: $f(x) = \frac{1}{1+e^{-x}}$

- Remember:

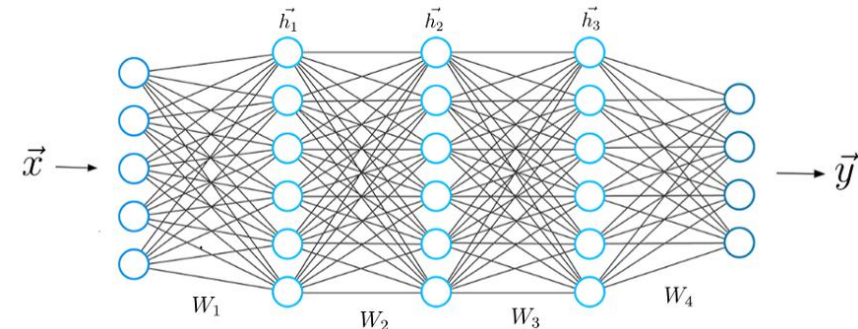
$$\vec{h}_1 = f(\vec{x} \cdot W_1)$$

$$\vec{h}_2 = f(\vec{h}_1 \cdot W_2)$$

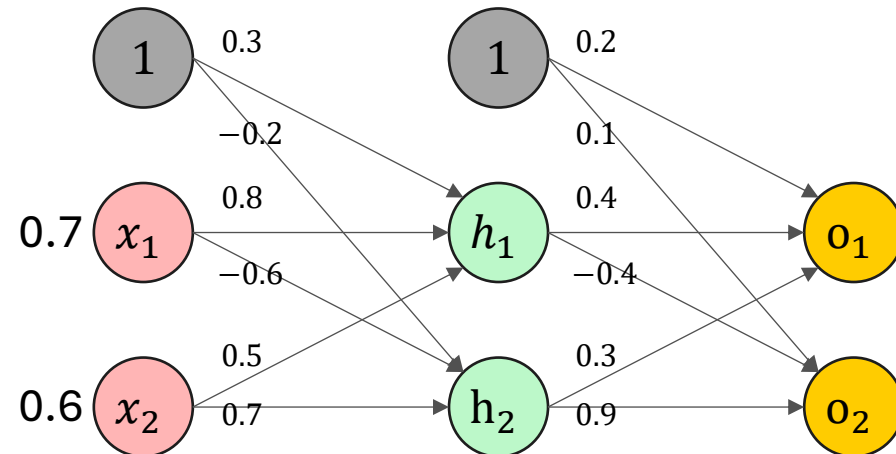
$$\vec{h}_3 = f(\vec{h}_2 \cdot W_3)$$

$$\vec{y} = f(\vec{h}_3 \cdot W_4)$$

$$\vec{y} = f(xw) = f^{(n)}(\dots f^{(2)}(f^{(1)}(\vec{x} \cdot W_1) \cdot W_2) \dots \cdot W_n)$$



- Let's apply this to our artificial neural network



Net input to h_1/h_2 :

weights	input	weighted input
$\begin{pmatrix} 0.3 & 0.8 & 0.5 \\ -0.2 & -0.6 & 0.7 \end{pmatrix}$	$\begin{pmatrix} 1 \\ 0.7 \\ 0.6 \end{pmatrix}$	$= \begin{pmatrix} 1.16 \\ -0.2 \end{pmatrix}$

Output h_1/h_2 :

$$f\left(\begin{pmatrix} 1.16 \\ -0.2 \end{pmatrix}\right) = \begin{pmatrix} 0.761 \\ 0.45 \end{pmatrix}$$

Output $o_1/o_2 = \vec{y}$:

Net input o_1/o_2 is computed equivalently

$$f\left(\begin{pmatrix} 0.639 \\ 0.2 \end{pmatrix}\right) = \begin{pmatrix} 0.655 \\ 0.55 \end{pmatrix}$$

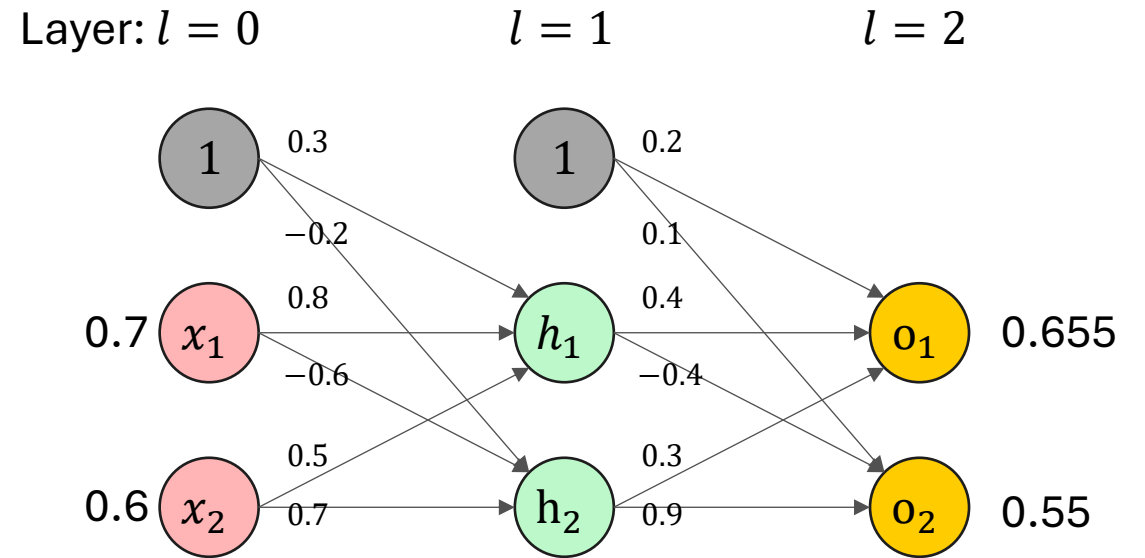
The output of each layer is also referred to as its respective **activation level**.

3. CALCULATE THE (OUTPUT) ERROR

Calculate the error relating expected and received output

- With the network output $\mathbf{y}$ that resulted from propagating the signal through the network, we can now calculate the **error signal**. Thus, if the l -layer is our output layer, we get the **observed error** for an input as $\delta^{(l)} = o^{(l)} - y$ with o
- The idea is to **propagate an error signal δ (computed in a single teaching step) back to the $(l-1)$ -layer**, i.e., neurons whose output signal represents the input for the neuron just discussed.
- Note that we **so far** simplified error computation in two ways
 - We compute the error for one sample only. In realistic scenarios, we do this batch-wise, i.e., for a small subset of our training data.
 - The error computation is just the difference between the expected and the predicted output. In realistic scenarios we rather use
 - MSE, MAE, ... (regression)
 - Cross-Entropy Loss, ... (classification)
- Note, according to the books this is the last step before the actual backpropagation algorithm starts.

See introduction -> metrics



Output $o_1/o_2 = \vec{y} = \begin{pmatrix} 0.655 \\ 0.55 \end{pmatrix}$:

Exemplary expected output $o^{(2)} = \begin{pmatrix} 0.9 \\ 0.2 \end{pmatrix}$

Error $\delta^{(2)} = \begin{pmatrix} 0.9 \\ 0.2 \end{pmatrix} - \begin{pmatrix} 0.655 \\ 0.55 \end{pmatrix} = \begin{pmatrix} 0.245 \\ -0.35 \end{pmatrix}$

4. BACKPROPAGATION – DELTA VALUES

Remember our delta rule: $w_{ij}^{(l)} = w_{ij}^{(l)} + \Delta w_{ij}^{(l)}$,

with $\Delta w_{ij}^{(l)} = \eta(o_i - y_i) \underbrace{net_i}_{\delta_i^{(l)}}$ $\rightarrow x_i$ or $\sum_p w_{ip} h_p^{l-1}$
 For our example we set this to 1 weighted output of previous layer

Mathematical basis

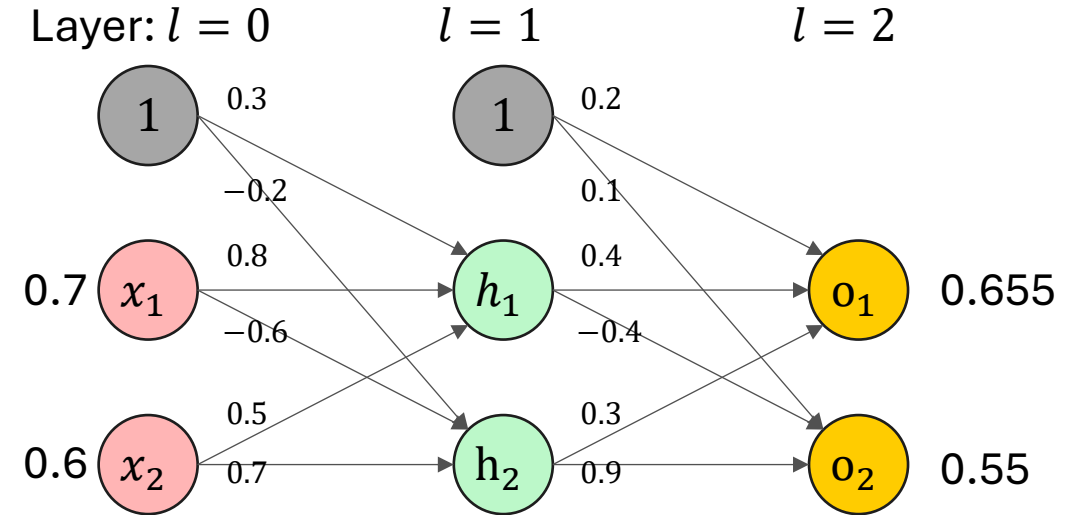
- We need to determine the necessary **change of our weights**.
- But we said that the **delta rule** does not help us too much here, because we do not know the expected outputs of hidden layers!
- So, what to do? The main problem is that we cannot compute the **error signal** $\delta^{(l)}$ for non-output layers.
- Solution: **Distinguish** between output layer and non-output layers

$$\delta_i^{(l)} = \begin{cases} f'(net_i)(o_i - y_i) & , \text{if output layer} \\ f'(net_i) \sum_k (\delta_k^{(l+1)} w_{ki}), & \text{else} \end{cases}$$

Sum error signals over all neurons of the next layers (this way, we recursively propagate the error backwards!)

- What is f' ? The derivative of the activation function!

$$f(x) = \frac{1}{1+e^{-x}}, \text{ i.e. } f'(x) = f(x)(1 - f(x))$$



So, first of all: Calculate all $\delta_i^{(l)}$!

$$\delta_1^{(2)} = \delta(o_1)$$

$$\delta_2^{(2)} = \delta(o_2)$$

$$\delta_1^{(1)} = \delta(h_1)$$

$$\delta_2^{(1)} = \delta(h_2)$$

$$\text{Error } \delta^{(2)} = \begin{pmatrix} 0.9 \\ 0.2 \end{pmatrix} - \begin{pmatrix} 0.655 \\ 0.55 \end{pmatrix} = \begin{pmatrix} 0.245 \\ -0.35 \end{pmatrix}$$

4. BACKPROPAGATION – DELTA VALUES

$$\delta_i^{(l)} = \begin{cases} f'(net_i)(o_i - y_i) \\ f'(net_i) \sum_k (\delta_k^{(l+1)} w_{ki}) \end{cases}$$

$$f(x) = \frac{1}{1+e^{-x}}, \text{ i.e. } f'(x) = f(x)(1-f(x))$$

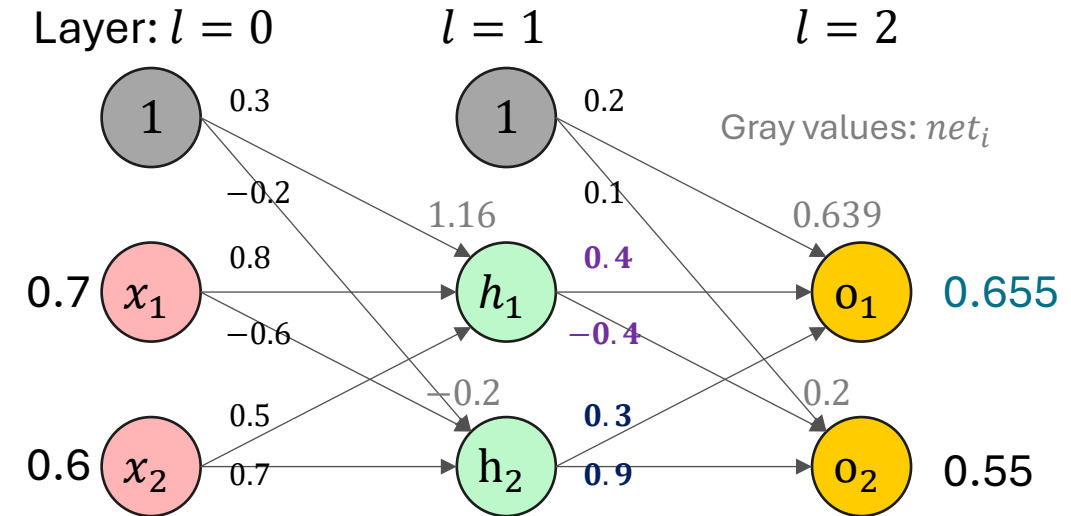
So, first of all: Calculate all $\delta_i^{(l)}$!

$$\delta_1^{(2)} = \delta(o_1) = f'(0.639)(0.245) = 0.655 \cdot (1 - 0.655)(0.245) = 0.055$$

$$\delta_2^{(2)} = \delta(o_2) = f'(0.2)(-0.35) = \dots = -0.087$$

$$\begin{aligned} \delta_1^{(1)} = \delta(h_1) &= f'(1.16)(\delta(o_1) \cdot 0.4 + \delta(o_2) \cdot (-0.4)) \\ &= 0.182(0.022 + 0.035) \\ &= 0.010 \end{aligned}$$

$$\begin{aligned} \delta_2^{(1)} = \delta(h_2) &= f'(-0.2)(\delta(o_1) \cdot 0.3 + \delta(o_2) \cdot 0.9) \\ &= 0.248(0.017 - 0.078) \\ &= -0.015 \end{aligned}$$



$$\text{Error } \delta^{(2)} = \begin{pmatrix} 0.9 \\ 0.2 \end{pmatrix} - \begin{pmatrix} 0.655 \\ 0.55 \end{pmatrix} = \begin{pmatrix} 0.245 \\ -0.35 \end{pmatrix}$$

4. BACKPROPAGATION – PROPAGATE THE ERROR

$$\begin{pmatrix} \delta(o_1) \\ \delta(o_2) \end{pmatrix}$$

=

$$\Delta w_{ij}^{(l)} = \eta(o_i - y_i) \text{net}_i^{(l)}$$

equals output of Layer l-1

Use the error signals to update the weights

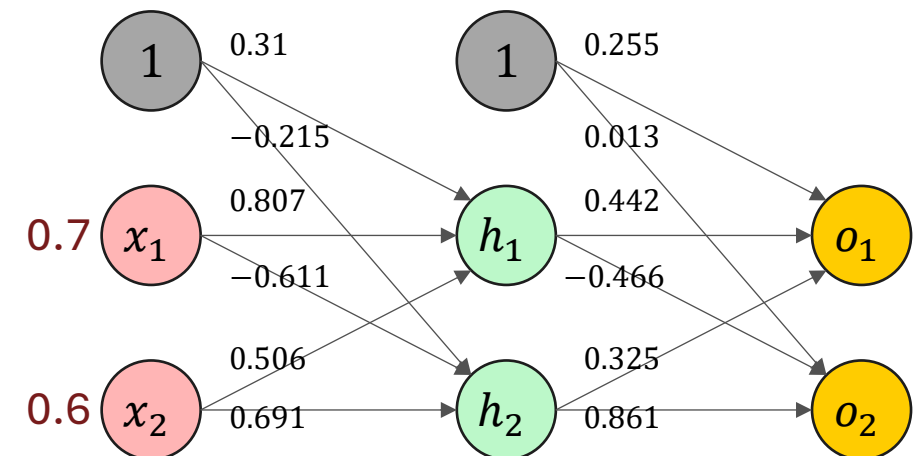
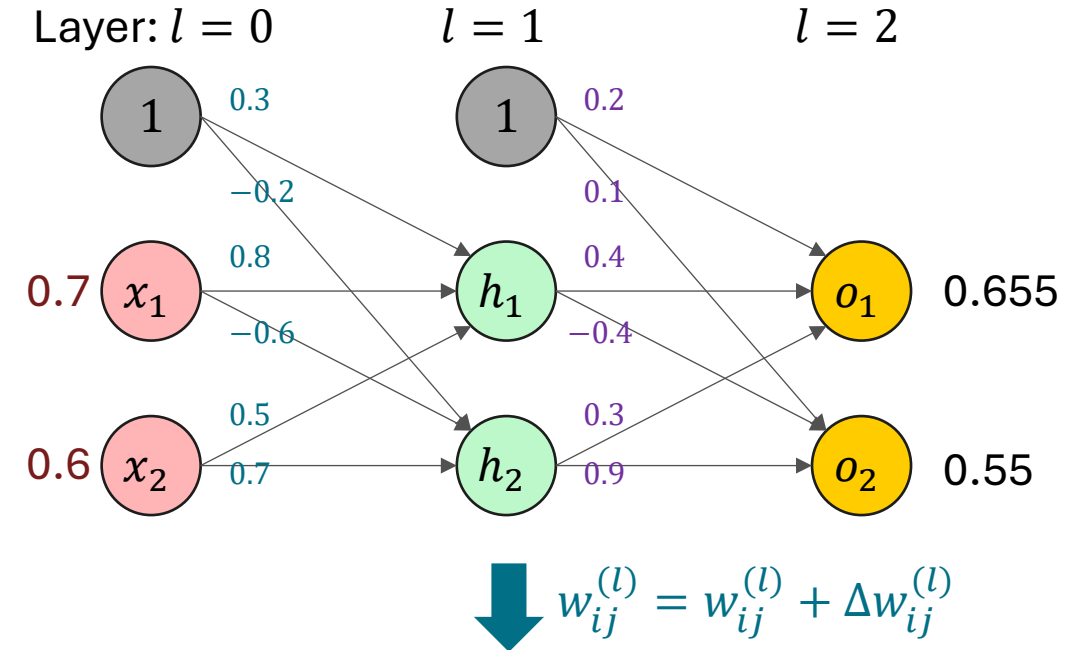
- Calculate the weight changes

$$\begin{aligned} \Delta w^{(2)} &= \begin{pmatrix} \delta_1^{(2)} \\ \delta_2^{(2)} \end{pmatrix} \begin{pmatrix} 1 & \text{net}_1^{(2)} & \text{net}_2^{(2)} \end{pmatrix} \\ &= \begin{pmatrix} 0.55 \\ -0.087 \end{pmatrix} \begin{pmatrix} 1 & y_{h1} & y_{h2} \end{pmatrix} \\ &= \begin{pmatrix} 0.055 & 0.042 & 0.025 \\ -0.0868 & -0.066 & -0.039 \end{pmatrix} \end{aligned}$$

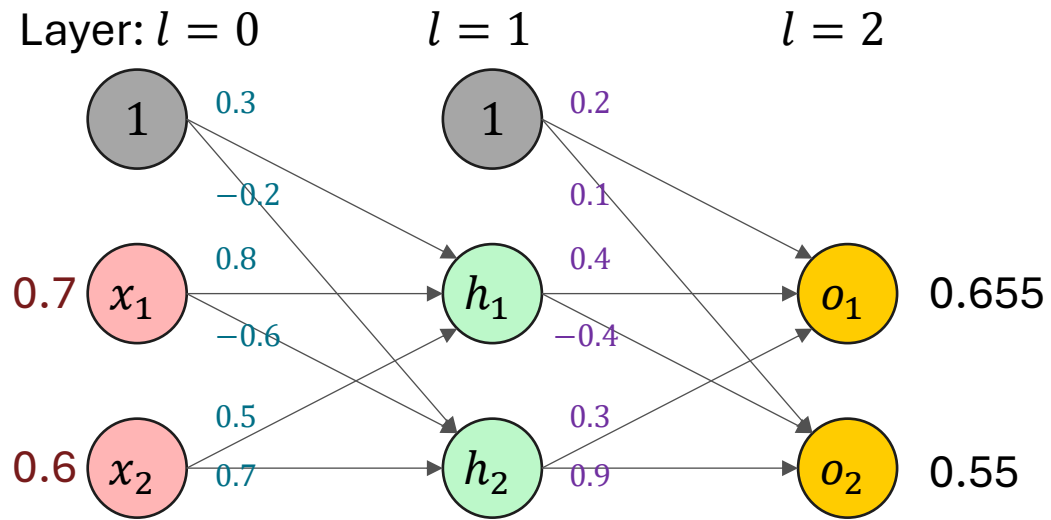
Why all positive?
Why all negative?

$$\begin{aligned} \Delta w^{(1)} &= \begin{pmatrix} \delta_1^{(1)} \\ \delta_2^{(1)} \end{pmatrix} \begin{pmatrix} 1 & \text{net}_1^{(1)} & \text{net}_2^{(1)} \end{pmatrix} \\ &= \begin{pmatrix} 0.010 \\ -0.015 \end{pmatrix} \begin{pmatrix} 1 & 0.7 & 0.6 \end{pmatrix} \\ &= \begin{pmatrix} 0.01 & 0.007 & 0.006 \\ -0.015 & -0.011 & -0.009 \end{pmatrix} \end{aligned}$$

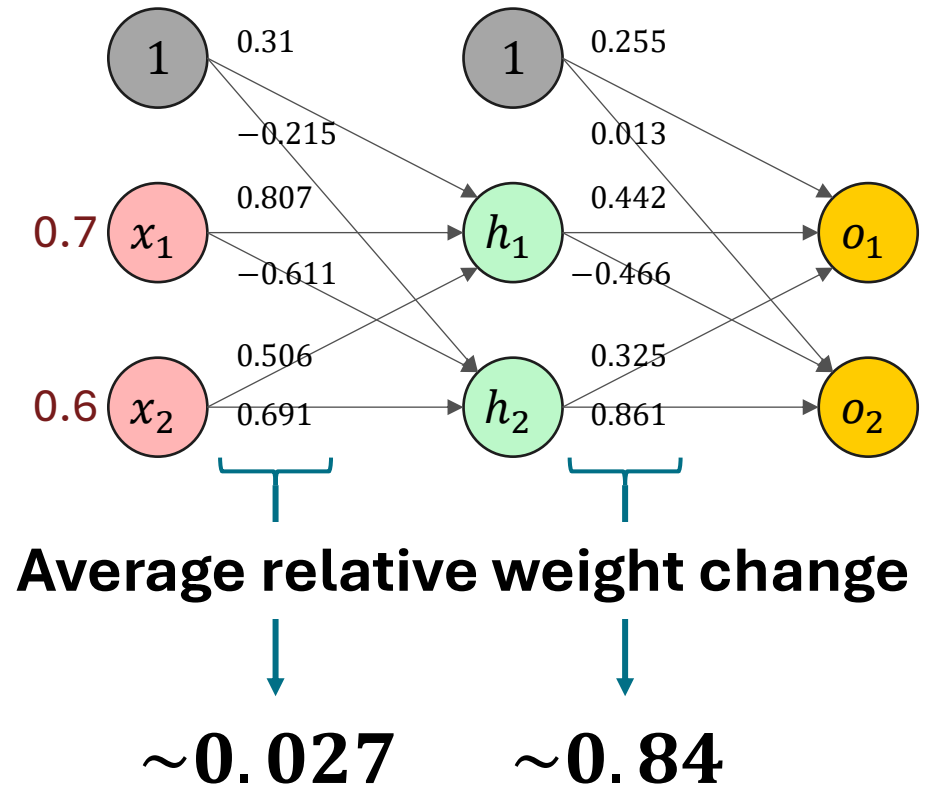
- Adjust weights



INTERESTING FINDING



$$w_{ij}^{(l)} = w_{ij}^{(l)} + \Delta w_{ij}^{(l)}$$



A COMMENT ON ERROR ERROR CALCULATION

- What we know: Like when training TLUs, we can produce an error δ from the network output y_u for a given neuron u and training sample $s \in S$ which contains an input i_u and target (desired output) o_u
$$\delta_u^{(s)} = o_u^{(s)} - y_u^{(s)}$$

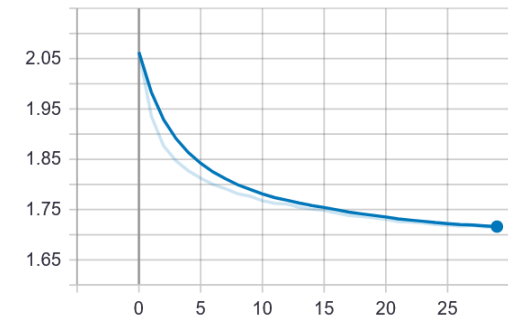
- Thus, so far, we calculated the error for a single data sample. However, in practical applications, the error is calculated for **a batch of samples**. But the question is how the error is calculated then.
- Simply summing up the error for all neurons $u \in U_{out}$, (where U_{out} is the set of output neurons) **can lead to errors canceling out each other**. Squaring the error solves that problem (and is the basis for measures like, for instance, the mean squared error):

$$\delta_u^{(s)} = \left(o_u^{(l)} - y_u^{(l)}\right)^2$$

- Training should terminate, when the total error for the entire training set L reaches a specified value, defined by the user, so

$$\delta = \sum_{s \in S} \sum_{u \in U_{out}} \left(o_u^{(l)} - y_u^{(l)}\right)^2$$

- The development of that error is called **learning curve** and indicates how the training is progressing



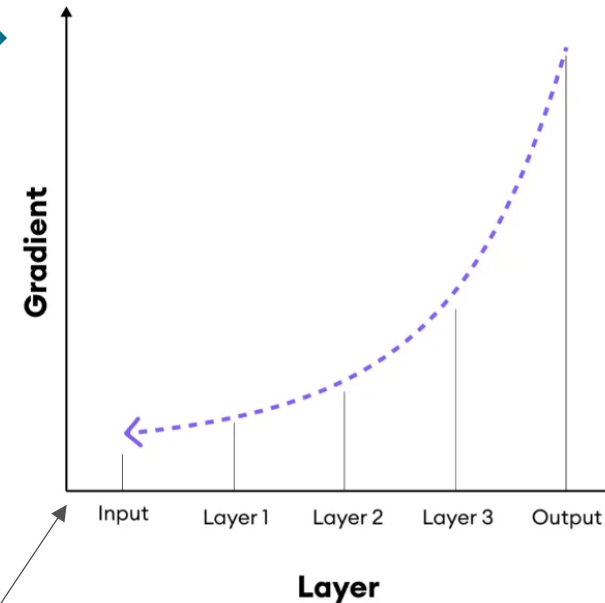
TYPICAL PROBLEMS WITH GRADIENT DESCENT – 1

VANISHING GRADIENT

Vanishing gradients occur when gradients become extremely small during backpropagation, causing early layers to learn very slowly or stop learning.

Causes:

- Activation functions with gradients $[0,1]$ in combination with deep neural networks.
- Example: **sigmoid function** σ
 - $0 \leq \sigma'(x) \leq \frac{1}{4}$... The gradient is 0.25 at maximum!
 - Output: $\vec{y} = f(xw) = f^{(n)}(\dots f^{(2)}(f^{(1)}(\vec{x} \cdot W_1) \cdot W_2) \dots \cdot W_n)$
 - Nested functions' derivatives are calculated using the chain rule of calculus, i.e., **Gradient computation** is defined as:
$$\frac{\partial y}{\partial W_1} = \frac{\partial y}{\partial a^{(n)}} \cdot \frac{\partial a^{(n)}}{\partial a^{(n-1)}} \cdot \frac{\partial a^{(n-1)}}{\partial a^{(n-2)}} \dots \frac{\partial a^{(1)}}{\partial W_1}$$
 and, thus, $\left| \frac{\partial y}{\partial W_1} \right| \leq 0.25^n C$
 - **This means that we multiply a values $[0,1]$ more often the more layers the neural network has! Thus, the gradient converges to 0!**
- Other cause: Very low weight initializations (regarding their absolute values) for the same reason (see gradient computation), low learning rates can amplify or mask this problem



Reason why ReLU helps much to mitigate the problem

If $f(x) = \max(0, x)$
Then: $f'(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{else} \end{cases}$

Thus, the gradient is not shrunk as it is passed backwards!
But it might cause dead neurons.

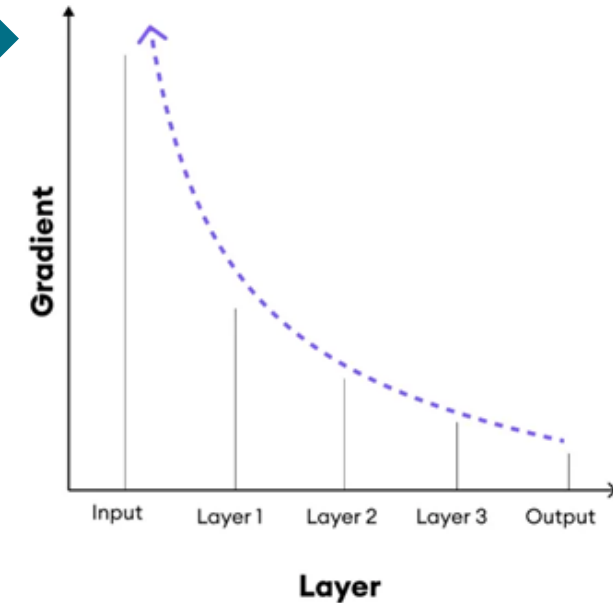
TYPICAL PROBLEMS WITH GRADIENT DESCENT – 2

EXPLODING GRADIENT

Exploding gradients occur when the gradients during backpropagation become too big, resulting in unstable training and possible divergence of the optimization process.

Causes:

- **Activation functions** with gradients > 1 (ReLU) in combination with deep neural networks. -> same argumentation as for vanishing gradient, i.e., the chain rule of calculus
- **Bad weight initialization (too high)**
- **High learning rates** can amplify or mask this problem



TYPICAL PROBLEMS WITH GRADIENT DESCENT – 3

a) Gradient descents converge against suboptimal minima

Every gradient descent procedure can get stuck within a **local minimum**. This problem is increasing proportionally to the size of the error surface, and there is no universal solution. In reality, one cannot know if the optimal minimum is reached and considers a training successful, if an acceptable minimum is found.

Typical Solution: Adjust learning parameter

b) Flat plateaus on the error surface may cause training slowness

When passing a flat **plateau**, the gradient also becomes negligibly small because there is hardly a descent what requires many further steps. A hypothetically possible gradient of 0 would completely stop the descent.

Typical Solution: Add a momentum term (an additional value decreasing with number of training steps)

c) Steep canyons in the error surface may cause oscillation

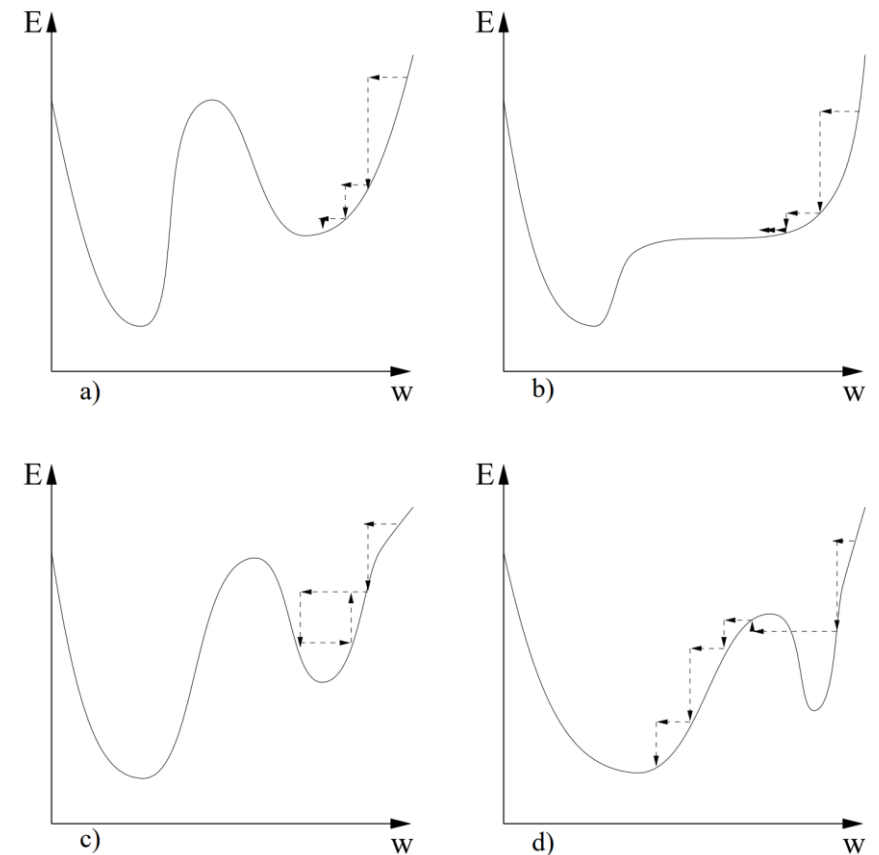
A sudden alternation from one very strong negative gradient to a very strong positive one can even result in **oscillation** (infrequent case).

Typical Solution: Add a momentum term

d) Even if good minima are reached, they may be left afterwards

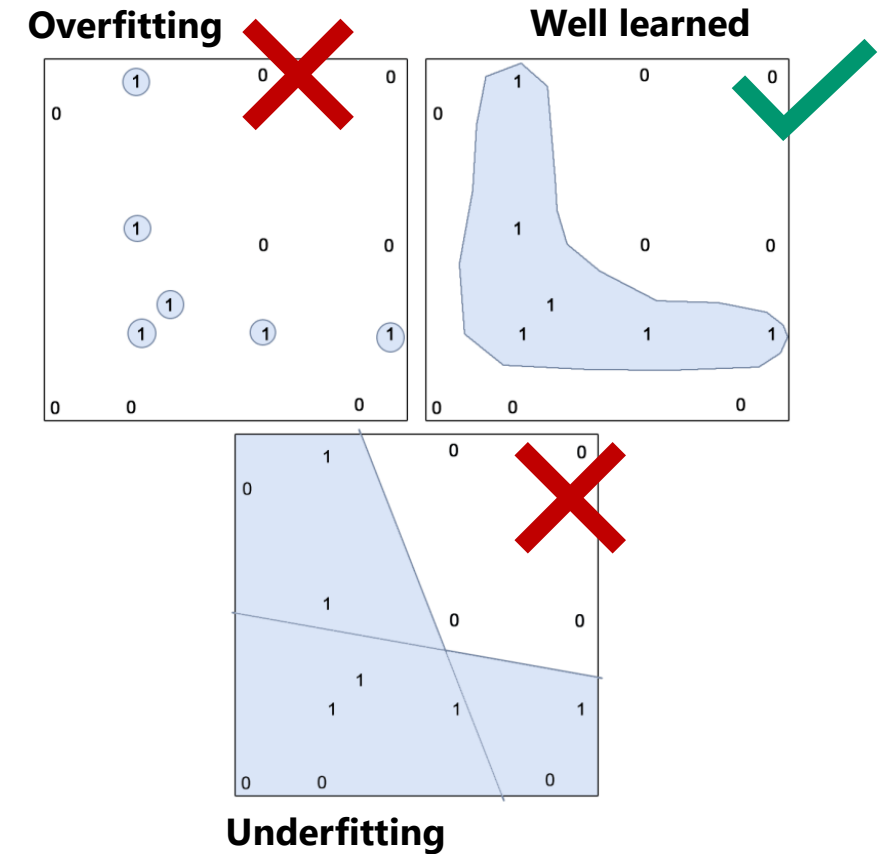
On the other hand, the gradient is very large at a steep slope so that large steps can be made, and a good minimum can possibly be **“skipped”**.

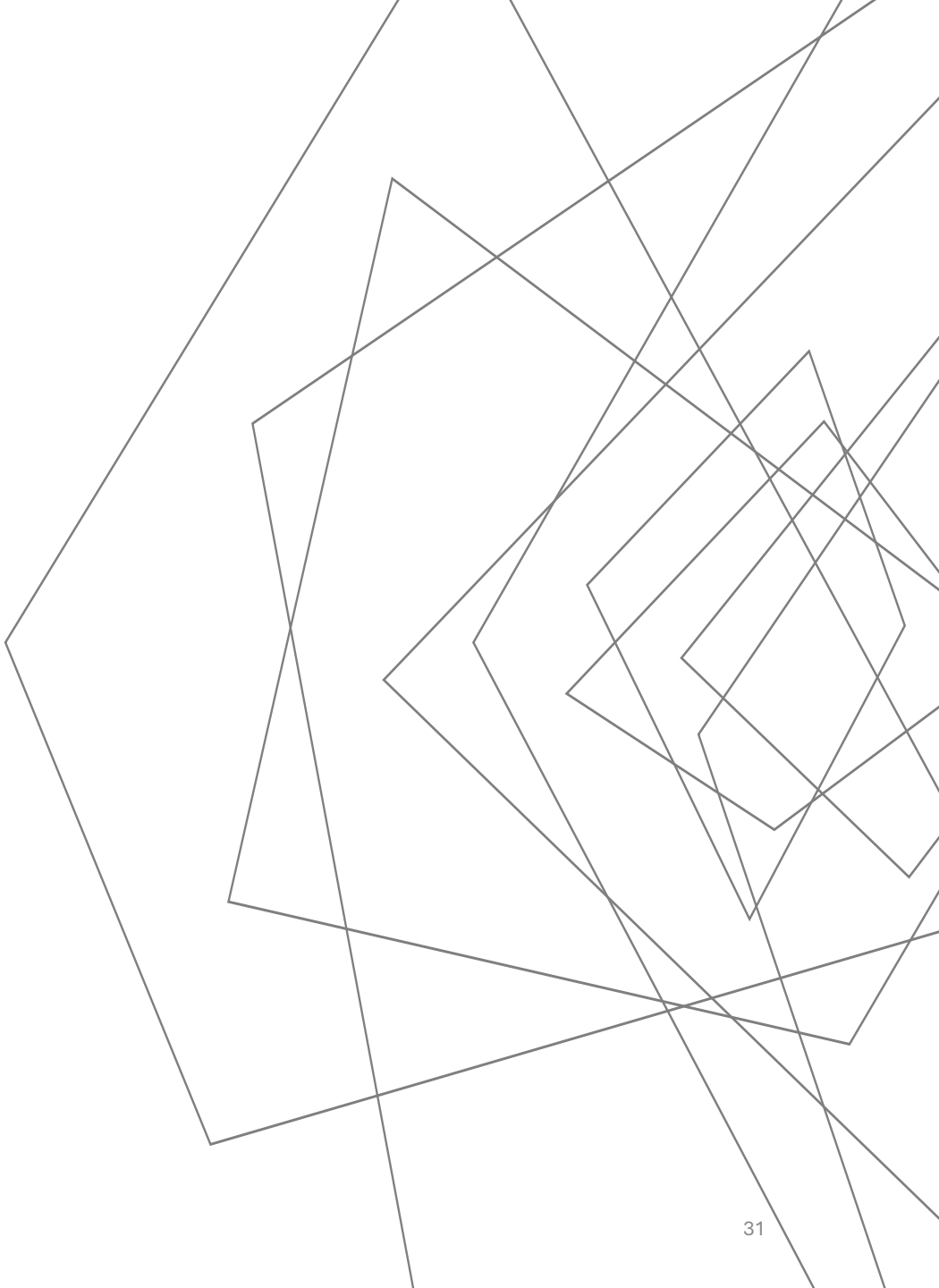
Typical Solution: Adjust learning parameter



TYPICAL PROBLEMS WITH GRADIENT DESCENT – 4

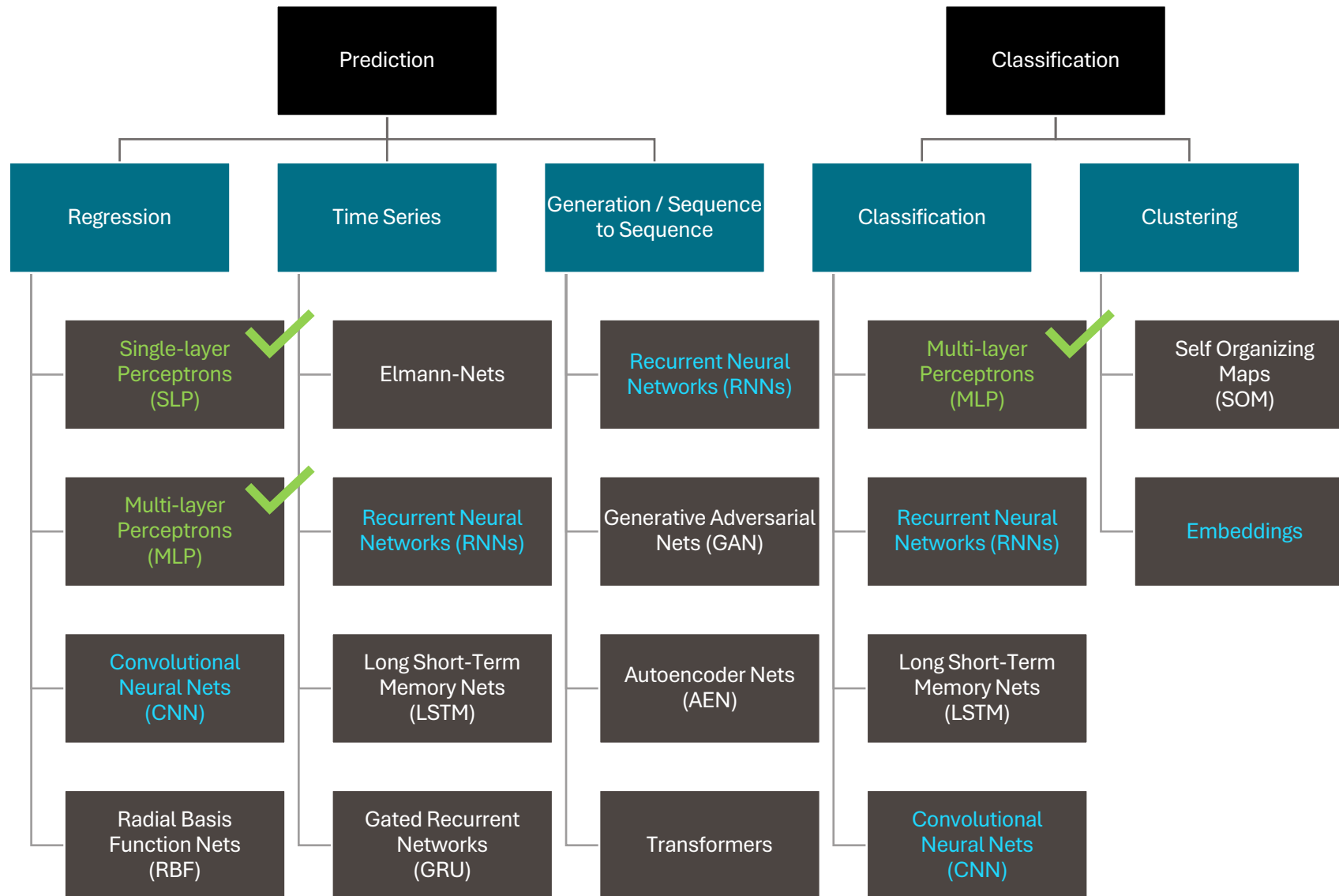
- Another possible, undesired outcome
 - After successful learning it's particularly interesting whether the network has only **memorized**, i.e. whether it uses our training samples to **exactly produce the right output (to this input)** but **provides wrong answers for all other problems** of the same class.
- Approach: **divide the set of training samples**
 - **Training set** used for training
 - **Test (verification) set** used for testing the quality
- The usual division
 - 70% for training data
 - 30% for verification data (randomly chosen)
- **Quality criterion**
 - We can finish the training when the network provides good results on the training data as well as on the verification data.





OUTLINE – WHAT’S NEXT?

ARCHITECTURES OF NEURAL NETWORKS



QUESTIONS?





DONE.

95028 Hof
Alfons-Goppel-Platz 1
Phone +49 9281 409-3000
Fax +49 9281 409-4000
mail@hof-university.de
www.hof-university.de